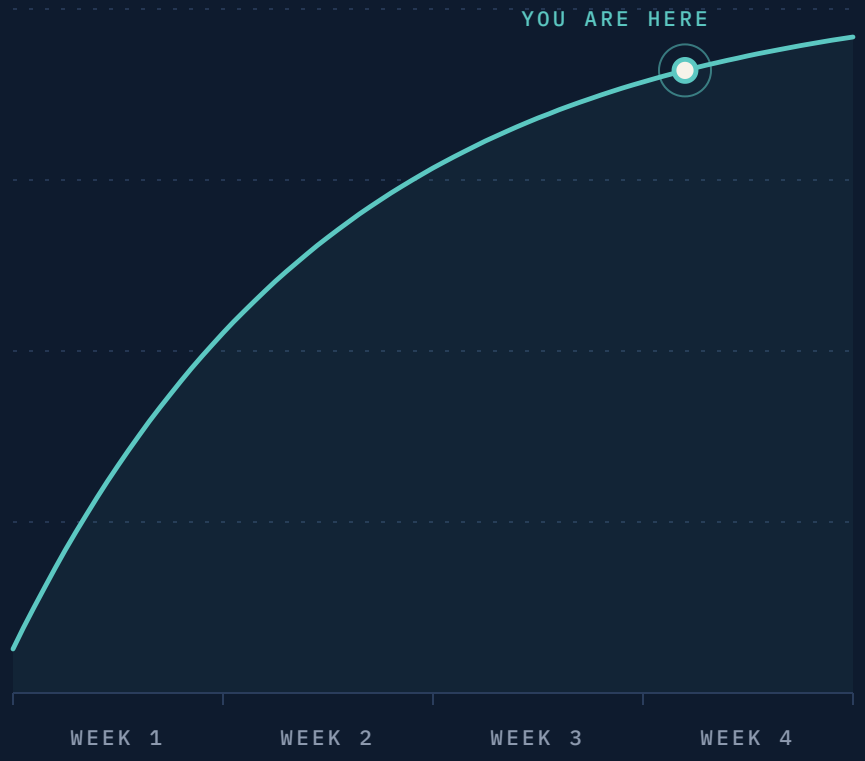
WEEK 4 · DAY 1 · OVERVIEW

Case Study

Consumer Credit



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 7 SECTIONS / 7 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Definition

We cover consumer lenders, and everyone quotes a public file of 30,000 card accounts. Before we do: does it match its documentation, and what share defaulted the next month? Which groups defaulted more, on how many accounts? Does that change with two groupings at once, or with columns we build: limit used, bill paid, months late? And which column says the most? Rates and counts in this file only: no causes, no lending advice.

ABOUT 55 MIN WITH EXERCISES / 7 SECTIONS / 7 EXERCISES

Your Route Through Day 1

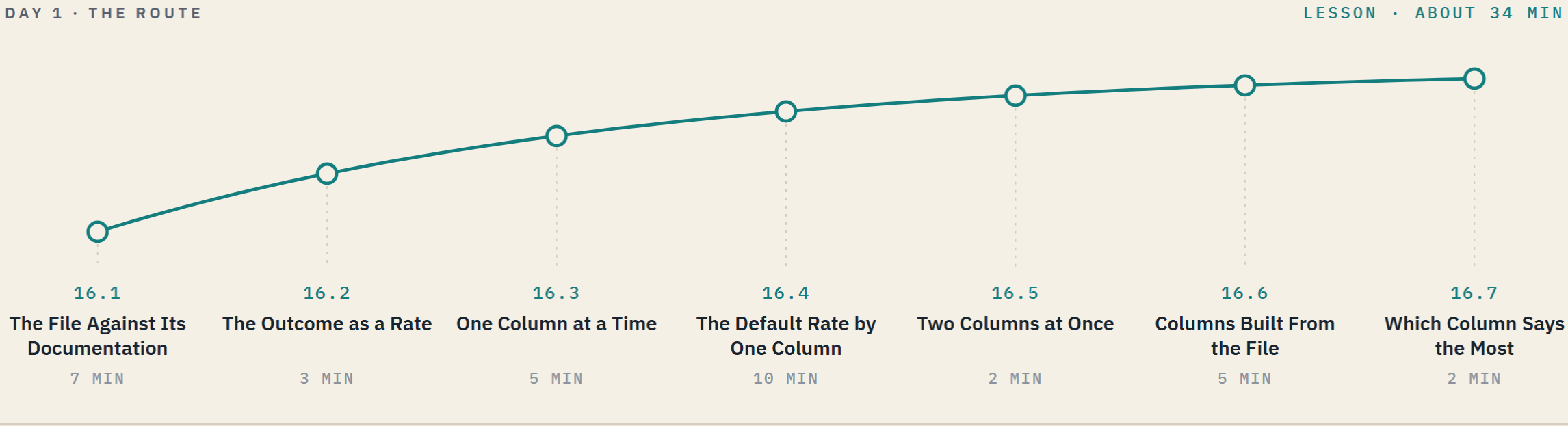


FIGURE 1 Seven sections, in order, with about how long each takes.

Time. About 34 minutes for the lesson, then about 21 for the seven exercises.

16.1 The File Against Its Documentation

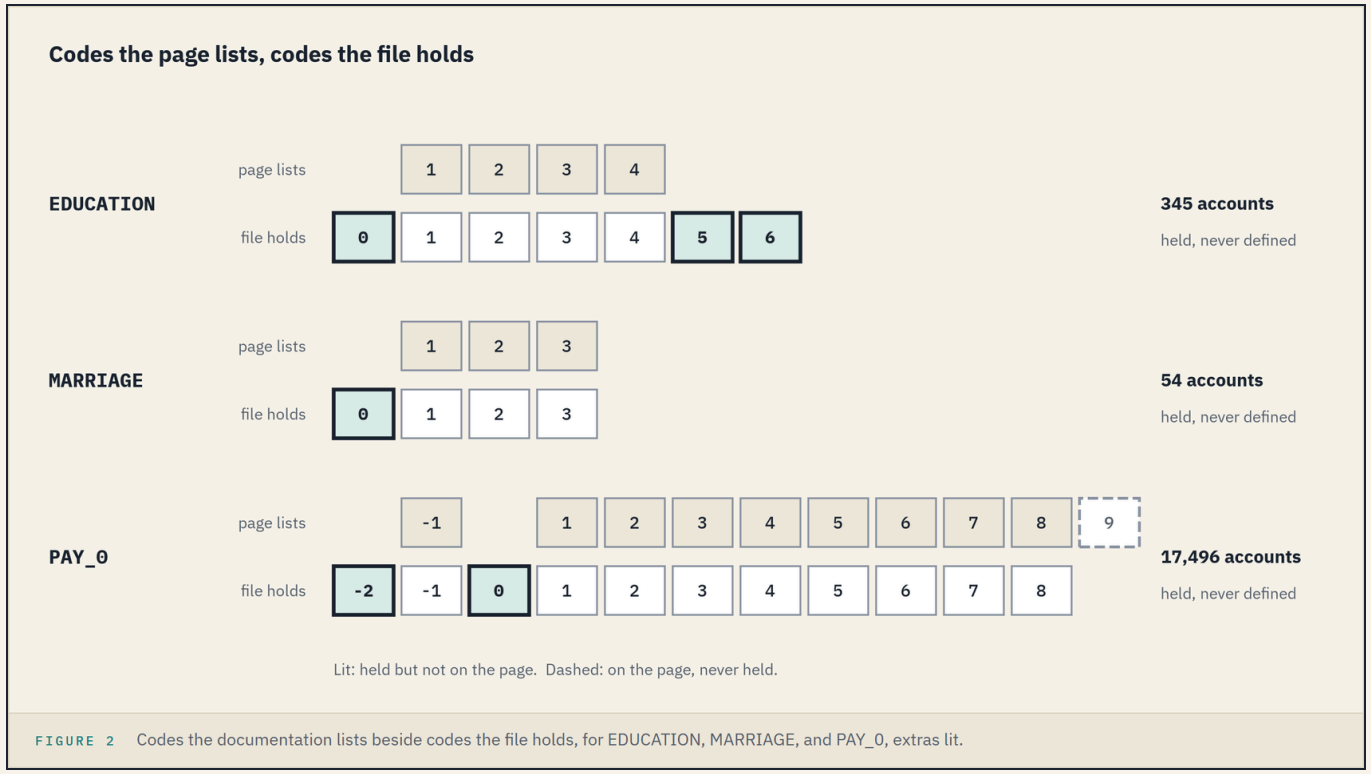


FIGURE 2 Codes the documentation lists beside codes the file holds, for EDUCATION, MARRIAGE, and PAY_0, extras lit.

REPAYMENT CODES, COUNTED

```
print('Largest code:', cards[pay_columns].max().max())
```

```
Largest code: 8
```

GAPS COUNTED

```
sex            0
education    345
marriage      54
dtype: int64
```

Nothing missing, nothing dropped: 345, 54, and 17,496 accounts kept under their own names.

16.2 The Outcome as a Rate

A RATE NEEDS ITS COUNT

```
code_zero = cards[cards['EDUCATION'] == 0]
print(round(1 / 14 * 100, 2), round(1 / 30000 * 100, 3))
```

```
14 accounts, 0 defaults
7.14 0.003
```

ONE LABEL PER OUTCOME

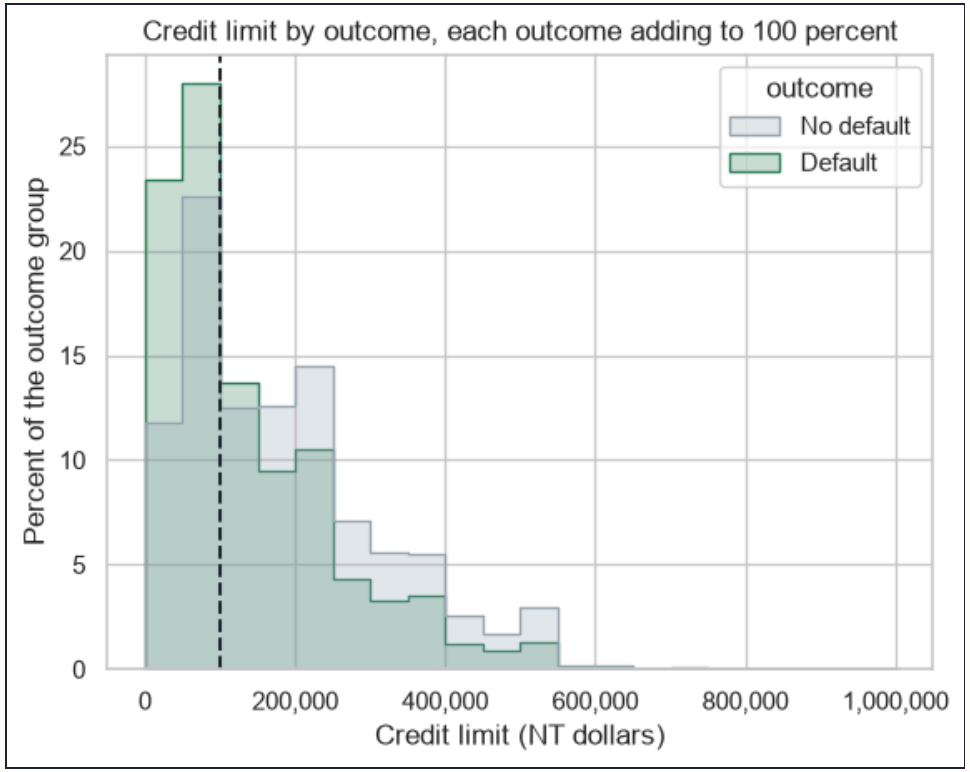
```
outcome_order = ['No default', 'Default']
cards['outcome'].value_counts()
```

```
outcome
No default    23364
Default        6636
Name: count, dtype: int64
```

Decision 4: one label, order, and colour per outcome. 22.12 percent, 6,636 of 30,000: now the columns, split by outcome.

16.3 One Column at a Time

LIMITS BY OUTCOME



SUMMARY, THEN A SHARE OF EACH GROUP

```
defaulted = cards[cards['default'] == 1]
no_default = cards[cards['default'] == 0]
```

```
Defaulted: 51.4 percent
No default: 34.4 percent
```

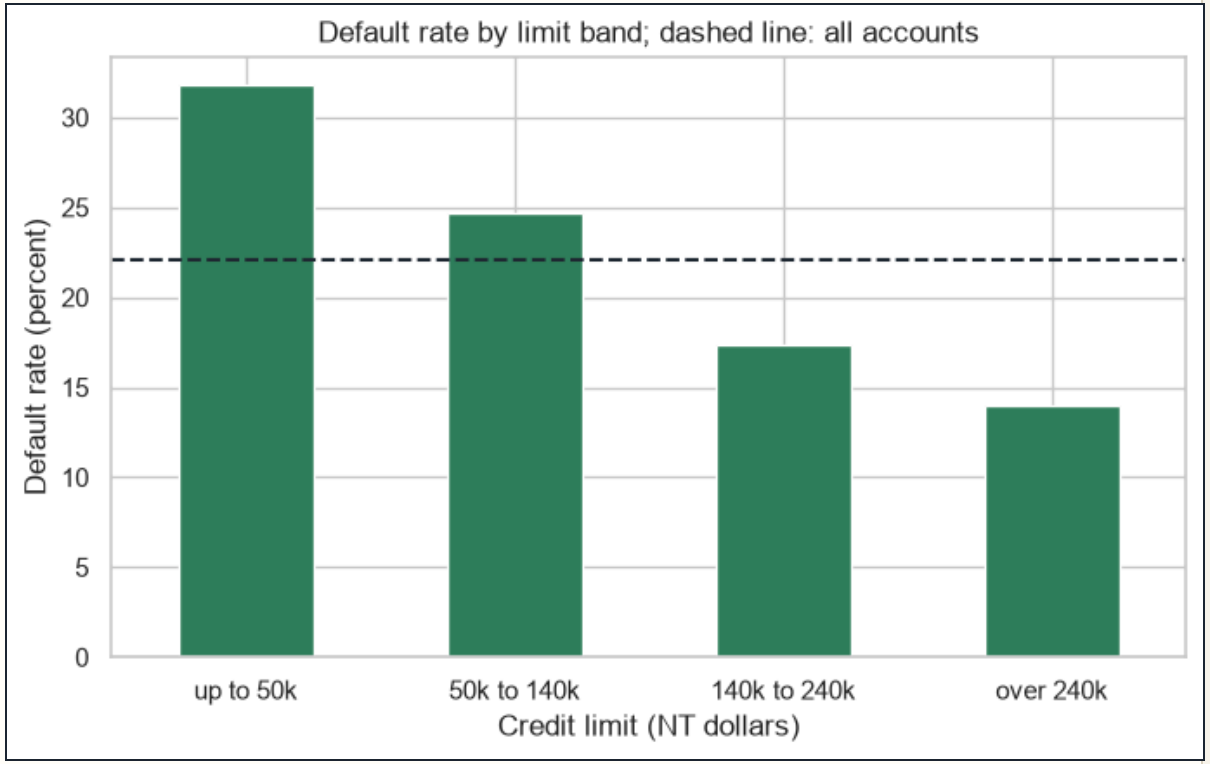
```
print(cards['LIMIT_BAL'].describe().round(0))
```

```
count      30000.0
mean      167484.0
std       129748.0
min        10000.0
25%        50000.0
50%       140000.0
75%       240000.0
max      1000000.0
Name: LIMIT_BAL, dtype: float64
```

Far is not wrong, as the EDA notebook drew it, and a share of a group is not a rate. Next, rates.

16.4 The Default Rate by One Column

RATE BY LIMIT BAND



RATE WITH ITS MARGIN

```
limit_rates = rate_table('limit_band')

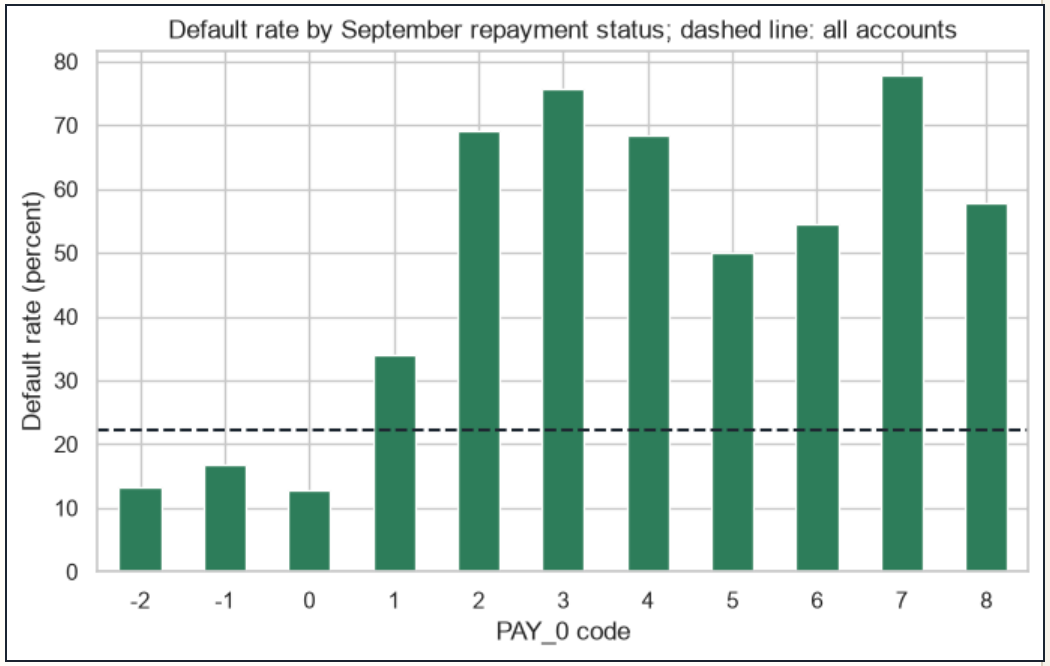
accounts  defaults  rate_pct  margin_pts
up to 50k       7676      2440     31.79        1.04
50k to 140k     7614      1882     24.72        0.97
140k to 240k    7643      1326     17.35        0.85
over 240k       7067       988     13.98        0.81
```

AGE IN FIVE BANDS

AGE_BAND	ACCOUNTS	DEFAULTS
21 to 30	11013	2471
31 to 40	10713	2189
41 to 50	6005	1399
51 to 60	1997	504
61 to 79	272	73

Repayment Status Against the Rest

RATE BY SEPTEMBER STATUS



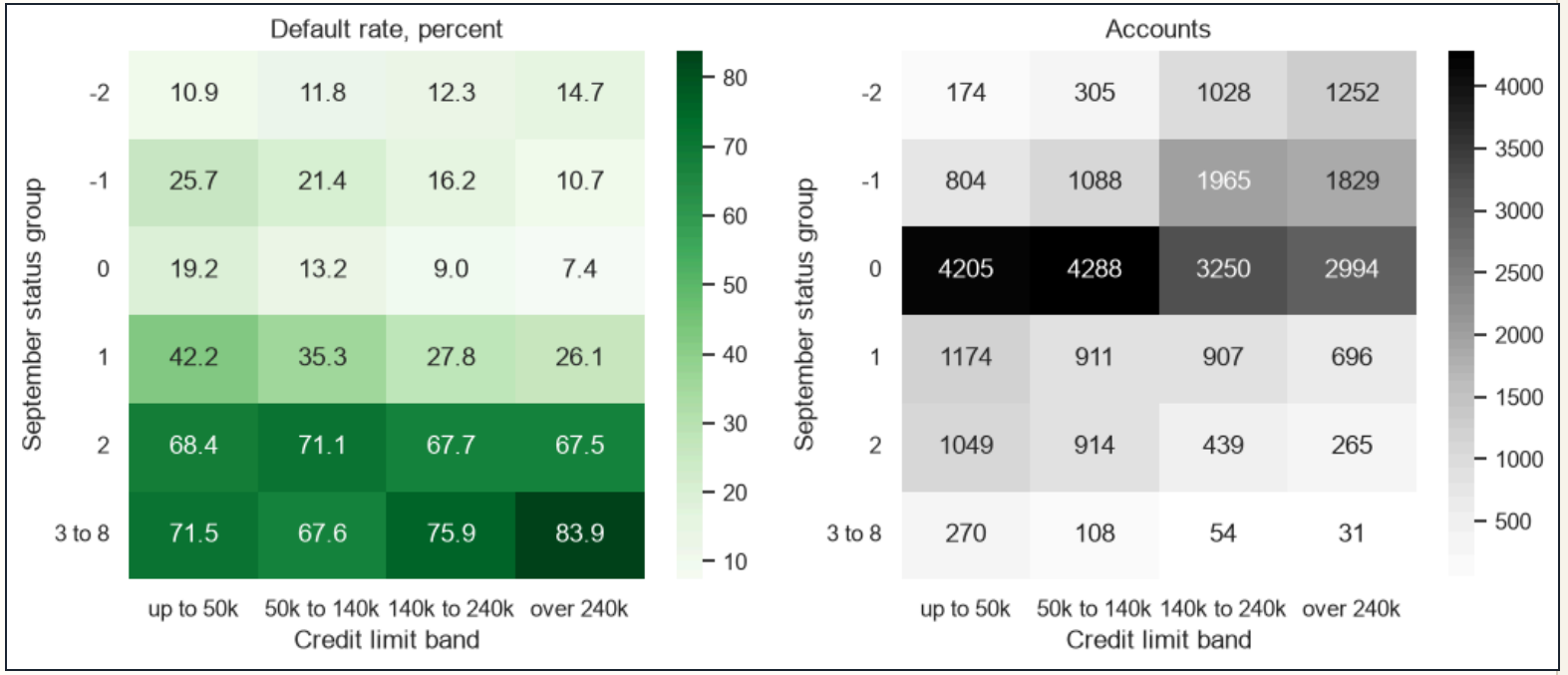
EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL	RESULT
Accounts	`count`	`=COUNTIF(C2:C30001, 1)`	11,888
Defaults	`sum`	`=SUMIF(C2:C30001, 1, Y2:Y30001)`	2,873
Rate	`mean`	`=AVERAGEIF(C2:C30001, 1, Y2:Y30001)`	0.2417
Bar range	`vmin=0, vmax=100`	Data Bars, More Rules, Minimum 0, Maximum 100	fixed bars
Age band	`pd.cut()`	`=VLOOKUP(F2, bands, 2, TRUE)`, on a table of lower edges	a label
Quartile edge	`pd.qcut()`	`=PERCENTILE.INC(B2:B30001, 0.25)`	50,000

Repayment status separates the most so far: 12.81 to 69.14 percent.

16.5 Two Columns at Once

RATE BESIDE COUNT



THE MARGIN OF A CELL

```
p = rate_pivot / 100

31 accounts: 12.9 points
4,288 accounts: 1.0 points
```

Read every rate with its cell's count.

16.6 Columns Built From the File

LIMIT IN USE, BY BAND

UTILISATION_BAND	ACCOUNTS	DEFAULTS	RATE_PCT
0 or below	2598	643	24.75
to 0.25	11440	1923	16.81
to 0.5	3588	780	21.74
to 0.75	3579	929	25.96
to 1	6680	1725	25.82
over 1	2115	636	30.07

SHARE OF THE BILL PAID, BY BAND

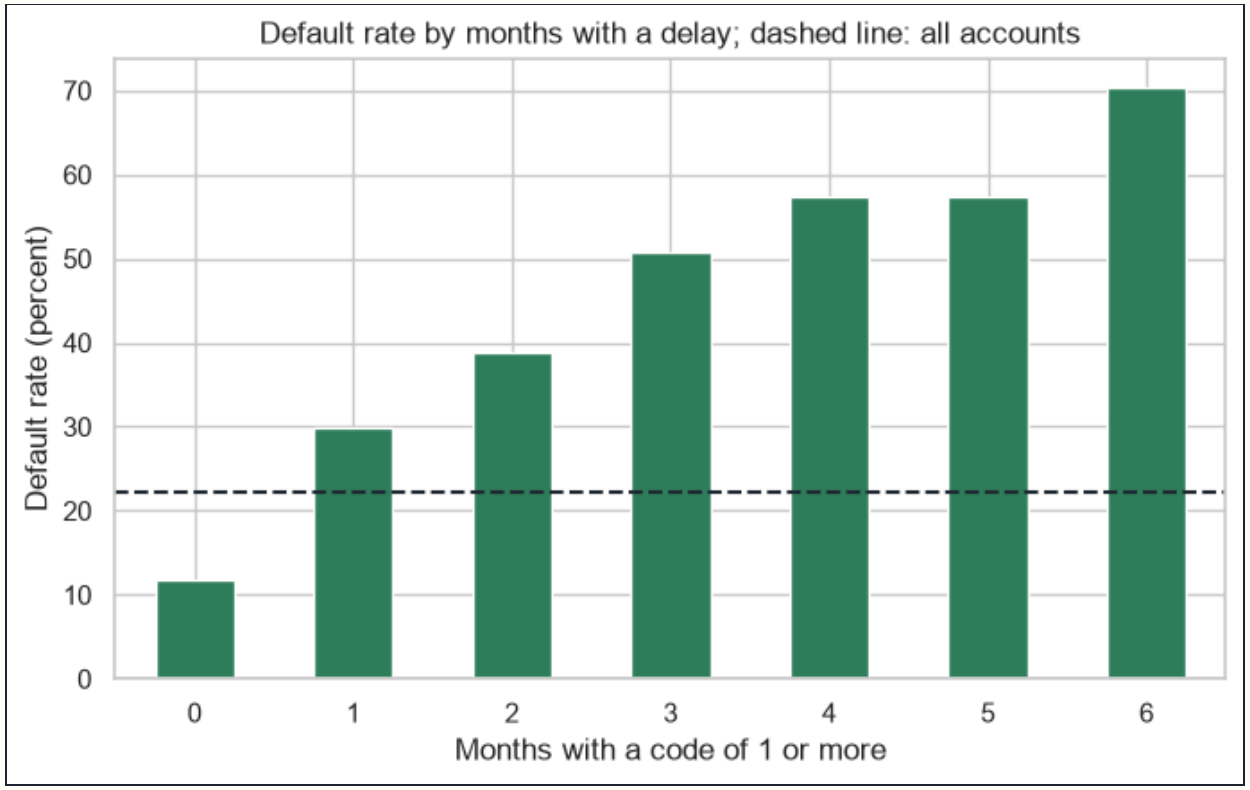
```
print(paid_rates)
print('Accounts in the table:', paid_rates['accounts'].sum())
```

```
accounts  defaults  rate_pct  margin_pts
under 0.05      11723      2989     25.50        0.79
0.05 to 0.1      5063      1107     21.86        1.14
0.1 to 0.5       5176      1084     20.94        1.11
0.5 to 1         1569       213     13.58        1.69
1 or more        3871       600     15.50        1.14
Accounts in the table: 27402
```

Three built columns; the months with a delay climb the steepest.

16.7 Which Column Says the Most

RATE BY MONTHS WITH A DELAY



WHAT THE NOTE ANSWERS

THE NOTE ASKED	ANSWER
Documentation	No gaps; undefined codes kept
Share defaulted	22.12 percent of 30,000
Groups	Status code 0, 12.81 percent; code 2, 69.14
Two groupings	83.9 percent, on 31 accounts
Built columns	11.71 to 70.32 percent by months delayed
Says the most	Months with a delay, 0.398

Months with a delay: 11.71 percent with none, 70.32 with six. The note is answered.

You Can Now

- Check a file against its documentation, code by code.
- Read the mean of a 0 and 1 column as a rate, quoted with its count.
- Cut bands with **`pd.cut()`** and **`pd.qcut()`**, and pivot rates beside counts.
- Build columns that survive a division by zero.

NEXT
WEEK 4 · DAY 2, PART 1

Analyzing Text, Part 1

Part 1, published statements become a table you can check.